claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_416cc0e5-a35\agent-a1226dfac63934679.jsonl`
- SHA-256: `b7971fd4cac250e0327a30fd057a761f117e7b9bf168698d0bdf4c647dc9d414`
- Source modified: `2026-06-22T09:52:17+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:49:24.928Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:

```
{
 "title": "MEDIUM: run_one_video() line 350 calls _segments_to_intervals(segs) unconditionally even when segs is None on an earlier error",
 "file": "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/scripts/nightly_reelcount.py:347-351",
 "severity": "medium",
 "detail": "If isinstance(result, Failure) at line 347, the function returns early at line 348. But if result is a Success with .value=None (a malformed segmenter), or result is a bare list=[], line 350 _segments_to_intervals([]) is called safely (empty list is OK). However, the re-run-once guard at line 362 compares count != baseline_count, where count is len(segs). If the first run crashed with err!=None (line 358), count would be None, and the rerun is skipped. This is correct (no quality metrics on error). No actual bug, but the logic path is subtle.",
 "fix": "No fix needed. The None-handling is correct (line 358 short-circuits before line 362)."
}
```

2. assistant (2026-06-22T09:49:27.041Z)

I'll analyze the nightly regression change by reading the specified files and verifying the finding about the `run_one_video()` function.

3. user (2026-06-22T09:49:28.130Z)

```
1	"""Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM nightly).
2
3	Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a deployment serves
4	it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does not change**
5	vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
```

```
of CACHED
trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real model*, so it
needs a GPU ? hence it runs on an ephemeral GCP GPU VM (``deploy/gcp/nightly_regression/``), emails the
owner the report, and self-deletes. It is **default-additive**: a tripwire, not the promotion gate.

What it asserts / reports, per the owner's ask ("quality metrics + regression report + cost"):
  ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected highlight clips.
    Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so this is asserted
    **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-None, see §guard).
  ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match block from
    ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small tolerance.
  ? COST of the run ? VM uptime (or processing wall) × machine $/hr via ``backend.billing`` ? USD + INR.

Extensible pattern (add a video = one manifest row + re-freeze):
  ``eval_baselines/reelcount_manifest.json``  ? the videos (gs:// / local / gdrive:// URIs) + segmenter
  ``eval_baselines/reelcount_baseline.json``  ? the frozen expected counts (regenerated, never hand-edited)

Usage
-----
Freeze the baseline from the current configured+checkpointed engine (after an intended change, or first run):
    python scripts/nightly_reelcount.py --update-baseline

Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails if --email):
    python scripts/nightly_reelcount.py --email

Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality dropped) · 2 operational
(manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter changed ? re-freeze).
"""
from __future__ import annotations

import argparse
import datetime as _dt
import json
import os
import sys
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional, Tuple

REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if REPO_ROOT not in sys.path:
    sys.path.insert(0, REPO_ROOT)

# billing is pure (typing only) ? safe to import at module top.
from backend import billing  # noqa: E402

DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")

```

```
#: Quality metrics we report + (when labels exist) gate ? higher is better, small tolerance.
QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
DEFAULT_QUALITY_TOL = 0.02     # quality is float/labeled ? a looser tripwire than the exact reel-count
DEFAULT_FX_INR_PER_USD = 83.0
#: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-south1, ~mid-2026).
DEFAULT_MACHINE_USD_PER_HR = 0.35


@dataclass(frozen=True)
class Tolerances:
    quality_tol: float = DEFAULT_QUALITY_TOL

    def to_dict(self) -> Dict[str, Any]:
        return {"quality_tol": self.quality_tol}


# --------------------------------------------------------------------------- #
# Pure cost math (wraps backend.billing ? no IO).
# --------------------------------------------------------------------------- #
def cost_report(billed_seconds: float, machine_usd_per_hr: float,
                fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
    """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing`` arithmetic).

    ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes ``--vm-uptime-seconds``;
    otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays for boot/install)."""
    rate_per_sec = float(machine_usd_per_hr) / 3600.0
    usage = {billing.WALL_SECONDS: float(billed_seconds)}
    rates = {billing.WALL_SECONDS: rate_per_sec}
    usd, breakdown = billing.compute_cost_usd(usage, rates)
    return {
        "billed_seconds": round(float(billed_seconds), 1),
        "gpu_seconds": round(float(gpu_seconds), 1),
        "machine_usd_per_hr": float(machine_usd_per_hr),
        "usd": usd,
        "inr": billing.to_inr(usd, fx_inr_per_usd),
        "breakdown": breakdown,
    }


# --------------------------------------------------------------------------- #
# Pure summary + comparison core (no IO ? unit-testable with plain dicts).
# --------------------------------------------------------------------------- #
def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
                      cost: Dict[str, Any]) -> Dict[str, Any]:
    """Compact, comparable snapshot from per-video run results.

    Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None, wall_seconds}``.
    A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it, never hides it)."""
    per_video: Dict[str, Any] = {}
    for r in run_results:
        per_video[r["name"]] = {
            "reel_count": r.get("reel_count"),
            "ok": bool(r.get("ok", False)),
            "error": r.get("error"),
            "quality": r.get("quality"),
        }
    return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
```

```
            "n": len(per_video)}


@dataclass
class Finding:
    video: str
    metric: str                 # "reel_count" | "error" | a quality key | "segmenter" | "corpus"
    kind: str                   # "regression" | "improvement" | "stale"
    baseline: Optional[float] = None
    current: Optional[float] = None
    detail: str = ""

    def message(self) -> str:
        if self.kind == "stale":
            return f"[STALE] {self.metric}: {self.detail}"
        if self.metric == "error":
            return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
        b = "?" if self.baseline is None else f"{self.baseline:g}"
        c = "?" if self.current is None else f"{self.current:g}"
        tag = "REGRESSION" if self.kind == "regression" else "improvement"
        return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"


@dataclass
class NightlyResult:
    findings: List[Finding] = field(default_factory=list)
    added: List[str] = field(default_factory=list)
    removed: List[str] = field(default_factory=list)

    @property
    def regressions(self) -> List[Finding]:
        return [f for f in self.findings if f.kind == "regression"]

    @property
    def improvements(self) -> List[Finding]:
        return [f for f in self.findings if f.kind == "improvement"]

    @property
    def stale(self) -> List[Finding]:
        return [f for f in self.findings if f.kind == "stale"]

    def overall_status(self) -> str:
        if self.regressions:
            return "REGRESSION"
        if self.stale:
            return "STALE"
        return "PASS"

    def exit_code(self) -> int:
        if self.regressions:
            return 1
        if self.stale:
            return 4
        return 0


def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) -> NightlyResult:
    """Diff current run vs frozen baseline.

    * Segmenter changed ? STALE (numbers not comparable; re-freeze).
    * Corpus (video set) changed ? STALE + report added/removed; still diff the INTERSECTION.
    * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR =
```

```
regression;
          quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
        """
        res = NightlyResult()
        if baseline.get("segmenter") != current.get("segmenter"):
            res.findings.append(Finding("", "segmenter", "stale",
                                        detail=f"{baseline.get('segmenter')} ? {current.get('segmenter')}"))
            return res

        b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
        res.added = sorted(set(c_pv) - set(b_pv))
        res.removed = sorted(set(b_pv) - set(c_pv))
        if res.added or res.removed:
            res.findings.append(Finding("", "corpus", "stale",
                                        detail=f"added={res.added} removed={res.removed}"))

        for v in sorted(set(b_pv) & set(c_pv)):
            b, c = b_pv[v], c_pv[v]
            # 1) pipeline must still succeed
            if not c.get("ok", False) or c.get("reel_count") is None:
                res.findings.append(Finding(v, "error", "regression",
                                            detail=str(c.get("error") or "no reel_count produced")))
                continue
            # 2) reel count ? EXACT
            bc, cc = b.get("reel_count"), c.get("reel_count")
            if bc is not None and cc is not None and cc != bc:
                res.findings.append(Finding(v, "reel_count", "regression", float(bc), float(cc)))
            # 3) quality metrics ? tolerance (only if both sides have them)
            bq, cq = b.get("quality") or {}, c.get("quality") or {}
            for m in QUALITY_HIGHER:
                bv, cv = bq.get(m), cq.get(m)
                if bv is None or cv is None:
                    continue
                if cv < bv - tols.quality_tol:
                    res.findings.append(Finding(v, m, "regression", bv, cv))
                elif cv > bv + tols.quality_tol:
                    res.findings.append(Finding(v, m, "improvement", bv, cv))
        return res


    # ------------------------------------------------------------------------------ #
    # Report formatting (pure).
    # ------------------------------------------------------------------------------ #
    def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
        if not qd or qd.get(key) is None:
            return "?"
        return f"{qd[key]:.3f}"


    def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
                      result: NightlyResult, date: str, head: str) -> str:
        status = result.overall_status()
        badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
                 "STALE": "? STALE (re-freeze the baseline)"}[status]
        cost = current.get("cost", {})
        L: List[str] = [
            f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
            "",
            f"**Status: {badge}**  ·  exit code {result.exit_code()}  ·  segmenter `{current.get('segmenter')}`",
            "",
            f"- HEAD `{head}`  ·  baseline `{baseline.get('git_commit', '?')}` (frozen
```

```
{baseline.get('generated_at', '?')})",
        f"- **Run cost: ${cost.get('usd', 0):.4f} (?{cost.get('inr', 0):.2f})**  ·  "
        f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr', 0)}/hr",
        "",
    ]
    if result.regressions:
        L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions] + [""]
    if result.stale:
        L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in result.stale]
        L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after confirming the change is intended).", ""]

    # Per-video table.
    b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
    L += ["## Per-video", "",
          "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
          "|---|---|---|---|---|"]
    for v in sorted(set(b_pv) | set(c_pv)):
        b, c = b_pv.get(v, {}), c_pv.get(v, {})
        bc = b.get("reel_count")
        cc = c.get("reel_count")
        rc = f"{'?' if bc is None else bc}?{'ERROR' if not c.get('ok', True) else ('?' if cc is None else cc)}"
        bq, cq = b.get("quality"), c.get("quality")
        tf = f"{_q(bq, 'tol_f1')}?{_q(cq, 'tol_f1')}"
        f5 = f"{_q(bq, 'f1@0.5')}?{_q(cq, 'f1@0.5')}"
        vstatus = "?" if any(f.video == v and f.kind == "regression" for f in result.findings) else "?"
        L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
    L.append("")

    if result.improvements:
        L += ["_Improvements over baseline (re-freeze to ratchet up):_"] \
            + [f"- {f.message()}" for f in result.improvements] + [""]
    L += ["---",
          "_Full configured+checkpointed pipeline on the real model (GCP GPU VM). Tripwire only ? does "
          "not change the promotion gate. Complements PR #202's cached-trajectory CPU re-score._"]
    return "\n".join(L)


# -------------------------------------------------------------------------- #
# IO shell ? running the configured pipeline per video (needs backend + GPU + video).
# -------------------------------------------------------------------------- #
def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
    """Load ground-truth rally intervals from a golden-features JSON (``gts`` key) or a rallies CSV
    (``start,end`` columns / pairs). Returns None when no labels are configured."""
    if not labels_ref:
        return None
    from backend.storage import fetch
    local = fetch(labels_ref)
    if local.endswith(".json"):
        with open(local, encoding="utf-8") as fh:
            data = json.load(fh)
        return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
    # CSV: header start,end (or first two numeric columns)
    import csv
    out: List[Tuple[float, float]] = []
    with open(local, encoding="utf-8") as fh:
        for row in csv.reader(fh):
```

```
            try:
                out.append((float(row[0]), float(row[1])))
            except (ValueError, IndexError):
                continue
    return out


def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
    """``(start, end)`` pairs from segmenter result dicts, tolerant of the two key conventions in the
    codebase (``start_time``/``end_time`` ? motion/DB shape ? or ``start``/``end``). Used only for the
    optional quality metrics; the reel COUNT is ``len(segs)`` regardless, so a key mismatch degrades
    quality scoring gracefully (skipped) without ever miscounting reels."""
    out: List[Tuple[float, float]] = []
    for s in segs:
        start = s.get("start_time", s.get("start"))
        end = s.get("end_time", s.get("end"))
        if start is not None and end is not None:
            out.append((float(start), float(end)))
    return out


def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
                  rerun_on_mismatch: bool = True,
                  baseline_count: Optional[int] = None) -> Dict[str, Any]:
    """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy backend imports.

    The re-run-once guard (the one known non-determinism ? a transient ``add_play_segment``?None drop,
    database.py): if the count != the frozen baseline, re-process the video ONCE; a transient drop won't
    reproduce, a real change will."""
    import random
    import time

    from backend.eval import rally_seg_eval
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters import segmenter_registry
    from backend.results import Failure
    from backend.storage import fetch
    from backend.utils.hashing import compute_video_id

    name = video["name"]
    try:
        local = fetch(video["uri"])
    except Exception as e:  # noqa: BLE001 ? surface, never hide
        return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed: {e}",
                "quality": None, "wall_seconds": 0.0}

    gts = _load_gts(video.get("labels_uri"))
    seg_cls = segmenter_registry.get(segmenter)
    if not seg_cls:
        return {"name": name, "reel_count": None, "ok": False,
                "error": f"segmenter '{segmenter}' not registered", "quality": None, "wall_seconds": 0.0}

    def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float, float]]], float, Optional[str]]:
        random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields are reproducible too
        db = Database(db_path)
```

```
        video_id = compute_video_id(local)
        t0 = time.monotonic()
        result = seg_cls(db, config).process_video(video_id, local)
        wall = time.monotonic() - t0
        if isinstance(result, Failure):
            return None, None, wall, getattr(result, "message", "segmenter failed")
        segs = result.value if hasattr(result, "value") else result
        intervals = _segments_to_intervals(segs)
        return len(segs), intervals, wall, None

    import tempfile
    total_wall = 0.0
    with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
        count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
        total_wall += wall
        if err is not None:
            return {"name": name, "reel_count": None, "ok": False, "error": err,
                    "quality": None, "wall_seconds": round(total_wall, 2)}
        # re-run-once guard for the transient DB-drop flake
        if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
            count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
            total_wall += wall2
            if err2 is None and count2 is not None:
                count, intervals = count2, intervals2  # the reproduced (or corrected) value

        quality = None
        if gts and intervals is not None:
            row = rally_seg_eval.score_intervals(intervals, gts)
            quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25", "tol_precision", "tol_recall")
                       if k in row}

    return {"name": name, "reel_count": count, "ok": True, "error": None,
            "quality": quality, "wall_seconds": round(total_wall, 2)}


def load_manifest(path: str) -> Dict[str, Any]:
    with open(path, encoding="utf-8") as fh:
        return json.load(fh)


def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
               rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
    """Run every manifest video through the configured pipeline. Returns (run_results, total_wall_s)."""
    from backend.config import load_config

    config = load_config()
    # apply manifest config overrides onto the typed config (dotted keys)
    for dotted, val in (manifest.get("config_overrides") or {}).items():
        _set_dotted(config, dotted, val)
    segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None), "default_segmenter", "motion")
    b_pv = (baseline or {}).get("per_video", {})
    results: List[Dict[str, Any]] = []
    total_wall = 0.0
    for v in manifest.get("videos", []):
        base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
        r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
        total_wall += float(r.get("wall_seconds") or 0.0)
        results.append(r)
    return results, total_wall
```

```python
def _set_dotted(config: Any, dotted: str, value: Any) -> None:
    """Best-effort dotted-path set onto a pydantic config (e.g. 'indexing.motion_threshold')."""
    parts = dotted.split(".")
    obj = config
    for p in parts[:-1]:
        obj = getattr(obj, p, None)
        if obj is None:
            return
    try:
        setattr(obj, parts[-1], value)
    except Exception:  # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't crash the run
        pass


def _git_head() -> str:
    import subprocess
    try:
        out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
                             capture_output=True, text=True, timeout=10)
        return out.stdout.strip() or "?"
    except Exception:  # noqa: BLE001
        return "?"


def load_baseline(path: str) -> Optional[Dict[str, Any]]:
    if not os.path.exists(path):
        return None
    with open(path, encoding="utf-8") as fh:
        return json.load(fh)


def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:
    out = dict(snapshot)
    out["generated_at"] = _dt.date.today().isoformat()
    out["git_commit"] = _git_head()
    out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python scripts/nightly_reelcount.py "
                   "--update-baseline` after an INTENDED engine change or when adding a video. Tied to the "
                   "configured+checkpointed engine + the manifest corpus.")
    out.pop("cost", None)  # cost is per-run, not part of the frozen baseline
    os.makedirs(os.path.dirname(path), exist_ok=True)
    tmp = path + ".tmp"
    with open(tmp, "w", encoding="utf-8") as fh:
        json.dump(out, fh, indent=2, sort_keys=True)
        fh.write("\n")
    os.replace(tmp, path)


def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count + quality + cost.")
    p.add_argument("--manifest", default=DEFAULT_MANIFEST)
    p.add_argument("--baseline", default=DEFAULT_BASELINE)
    p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
    p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
    p.add_argument("--vm-uptime-seconds", type=float, default=None,
                   help="authoritative VM uptime (boot?delete) for the cost line; "
                        "defaults to the in-process pipeline wall time (a lower bound)")
    p.add_argument("--no-rerun-guard", action="store_true",
                   help="disable the re-run-once-on-mismatch guard (the DB-drop flake mitigation)")
    p.add_argument("--update-baseline", action="store_true",
```

```
                   help="freeze the current reel counts + quality as the new baseline instead of checking")
    p.add_argument("--email", action="store_true", help="email the report (see scripts/nightly_email.py)")
    p.add_argument("--email-to", default=None, help="override the report recipient")
    p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD; for tests)")
    p.add_argument("--no-report", action="store_true", help="skip writing the dated report file")
    return p


def main(argv: Optional[List[str]] = None) -> int:
    import logging
    logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
    log = logging.getLogger("nightly_reelcount")
    args = build_parser().parse_args(argv)
    tols = Tolerances(quality_tol=args.quality_tol)
    date = args.date or _dt.date.today().isoformat()

    if not os.path.exists(args.manifest):
        log.error("manifest not found: %s", args.manifest)
        return 2
    manifest = load_manifest(args.manifest)
    baseline = load_baseline(args.baseline)

    try:
        run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not args.no_rerun_guard)
    except Exception as e:  # noqa: BLE001 ? operational failure (deps/config) ? exit 2, never a false "pass"
        log.error("run failed: %s", e)
        return 2
    if not run_results:
        log.error("no videos in manifest %s", args.manifest)
        return 2

    billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else total_wall
    cost = cost_report(
        billed_seconds=billed,
        machine_usd_per_hr=float(manifest.get("machine_usd_per_hr", DEFAULT_MACHINE_USD_PER_HR)),
        fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
        gpu_seconds=total_wall,
    )
    segmenter = manifest.get("segmenter", "motion")
    snapshot = summarize_results(run_results, segmenter, cost)
    snapshot["git_commit"] = _git_head()
    snapshot["generated_at"] = date

    if args.update_baseline:
        save_baseline(args.baseline, snapshot)
        log.info("baseline frozen ? %s", args.baseline)
        for r in run_results:
            log.info("  %s: reels=%s ok=%s", r["name"], r.get("reel_count"), r.get("ok"))
        return 0

    if baseline is None:
        log.error("no baseline at %s ? freeze one with --update-baseline (on the configured engine).", args.baseline)
        return 3

    result = compare(baseline, snapshot, tols)
```

```
    report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])

    if not args.no_report:
        os.makedirs(args.report_dir, exist_ok=True)
        md_path = os.path.join(args.report_dir, f"{date}.md")
        with open(md_path, "w", encoding="utf-8") as fh:
            fh.write(report + "\n")
        with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8") as fh:
            json.dump({"date": date, "status": result.overall_status(), "exit_code": result.exit_code(),
                       "snapshot": snapshot, "regressions": [f.message() for f in result.regressions]},
                      fh, indent=2, sort_keys=True)
        log.info("report ? %s", md_path)

    print(report)

    if args.email:
        try:
            from scripts.nightly_email import send_report
            subject = f"[KhelSutra nightly] {result.overall_status()} ? reel-count engine regression {date}"
            sent = send_report(subject, report, to=args.email_to)
            log.info("email %s", "sent" if sent else "skipped (no creds ? report still written)")
        except Exception as e:  # noqa: BLE001 ? a broken mailer must NOT mask the regression verdict
            log.error("email failed (report still written): %s", e)

    code = result.exit_code()
    if code == 1:
        log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions), args.report_dir)
    elif code == 4:
        log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-baseline")
    else:
        log.info("nightly reel-count check PASSED  (cost $%.4f / ?%.2f)", cost["usd"], cost["inr"])
    return code


if __name__ == "__main__":
    raise SystemExit(main())

```

4. user (2026-06-22T09:49:28.491Z)

```
"""Email the nightly regression report ? stdlib SMTP, creds from env or GCP Secret Manager.

Used by ``scripts/nightly_reelcount.py --email`` (and reusable by any nightly). Deliberately
dependency-light (stdlib ``smtplib`` + ``email.message``) and **graceful**: if no SMTP password can
be resolved it logs + returns ``False`` (the report is still written/printed ? a broken mailer must
never mask a regression).

Credential resolution (first hit wins), so it works on a GCP VM, locally, or in tests:
  ? Recipient:  ``--email-to`` arg  ?  ``NIGHTLY_EMAIL_TO``  ?  ``avi.dullu@gmail.com``
  ? SMTP host:  ``NIGHTLY_SMTP_HOST``  (default ``smtp.gmail.com``)   · port ``NIGHTLY_SMTP_PORT`` (587, STARTTLS)
  ? SMTP user:  ``NIGHTLY_SMTP_USER``  (default = recipient)
```

```
  ? SMTP pass:  ``NIGHTLY_SMTP_PASS``  ?  Secret Manager ``NIGHTLY_SMTP_SECRET``
                (``projects/<p>/secrets/<name>/versions/latest``; via the google-cloud-secret-manager
                 lib if installed, else the ``gcloud`` CLI). A Gmail **app password** is the simplest.

Set the secret once (owner):  echo -n '<app-password>' | gcloud secrets create nightly-smtp-pass --data-file=-
and on the VM:  export NIGHTLY_SMTP_SECRET=projects/<proj>/secrets/nightly-smtp-pass/versions/latest
                export NIGHTLY_SMTP_USER=<your-gmail> NIGHTLY_EMAIL_TO=avi.dullu@gmail.com
"""
from __future__ import annotations

import logging
import os
from email.message import EmailMessage
from typing import Optional

log = logging.getLogger("nightly_email")

DEFAULT_TO = "avi.dullu@gmail.com"
DEFAULT_HOST = "smtp.gmail.com"
DEFAULT_PORT = 587


def build_message(subject: str, body_md: str, from_addr: str, to_addr: str) -> EmailMessage:
    """A text/plain email carrying the markdown report (renders fine in any client). Pure ? testable."""
    msg = EmailMessage()
    msg["Subject"] = subject
    msg["From"] = from_addr
    msg["To"] = to_addr
    msg.set_content(body_md)
    return msg


def resolve_secret(env: Optional[dict] = None) -> Optional[str]:
    """Resolve the SMTP password from env or GCP Secret Manager. Returns None if unresolvable."""
    env = env if env is not None else os.environ
    direct = env.get("NIGHTLY_SMTP_PASS")
    if direct:
        return direct
    secret_ref = env.get("NIGHTLY_SMTP_SECRET")
    if not secret_ref:
        return None
    # Prefer the client lib; fall back to the gcloud CLI (no extra dependency required).
    try:
        from google.cloud import secretmanager  # type: ignore
        client = secretmanager.SecretManagerServiceClient()
        return client.access_secret_version(name=secret_ref).payload.data.decode("utf-8")
    except Exception:  # noqa: BLE001 ? lib absent or auth issue ? try the CLI
        pass
    try:
        import subprocess
        name = secret_ref.split("/secrets/", 1)[1].split("/", 1)[0] if "/secrets/" in secret_ref else secret_ref
        out = subprocess.run(["gcloud", "secrets", "versions", "access", "latest", "--secret", name],
                             capture_output=True, text=True, timeout=30)
        return out.stdout.strip() if out.returncode == 0 and out.stdout.strip() else
```

```
None
    except Exception as e:  # noqa: BLE001
        log.warning("secret-manager resolution failed: %s", e)
        return None


def send_report(subject: str, body_md: str, to: Optional[str] = None,
                env: Optional[dict] = None) -> bool:
    """Send the report. Returns True if sent, False if skipped (no creds) ? never raises on missing creds."""
    env = env if env is not None else os.environ
    to_addr = to or env.get("NIGHTLY_EMAIL_TO") or DEFAULT_TO
    host = env.get("NIGHTLY_SMTP_HOST", DEFAULT_HOST)
    port = int(env.get("NIGHTLY_SMTP_PORT", DEFAULT_PORT))
    user = env.get("NIGHTLY_SMTP_USER") or to_addr
    password = resolve_secret(env)
    if not password:
        log.warning("no SMTP password (set NIGHTLY_SMTP_PASS or NIGHTLY_SMTP_SECRET) ? email skipped.")
        return False

    import smtplib
    msg = build_message(subject, body_md, from_addr=user, to_addr=to_addr)
    with smtplib.SMTP(host, port, timeout=60) as s:
        s.starttls()
        s.login(user, password)
        s.send_message(msg)
    log.info("emailed report to %s", to_addr)
    return True

```

5. user (2026-06-22T09:49:29.374Z)

```
#!/usr/bin/env bash
# Startup-script for the NIGHTLY engine-regression VM. Runs UNATTENDED on boot, then DELETES the VM
# (cost rule: a running GPU VM is the only real cost). Survives SSH drops (it's a startup-script, not
# an interactive session). Reads its config from instance metadata (set by launch.sh).
#
# Flow:  Ops Agent  ?  pull repo @ HEAD from GCS  ?  build venv + WASB env  ?  stage weights  ?
#         run scripts/nightly_reelcount.py on the gs:// golden clips (emails the report) ?
#         upload report to GCS  ?  SELF-DELETE (even on failure, via the EXIT trap).
#
# Metadata keys (launch.sh sets these): repo-tgz-uri, git-sha, gcs-bucket, report-prefix,
#   weights-uri, email (true/false). NOT verified in CI ? dry-run via launch.sh on a real VM first.
set -uo pipefail
exec > >(tee -a /var/log/nightly-regression.log) 2>&1
echo "===== nightly-regression startup $(date -u +%FT%TZ) ====="
BOOT_EPOCH=$(date +%s)

md() { curl -s -H "Metadata-Flavor: Google" "http://metadata.google.internal/computeMetadata/v1/$1"; }
NAME=$(md instance/name)
ZONE=$(md instance/zone | awk -F/ '{print $NF}')
REPO_TGZ=$(md instance/attributes/repo-tgz-uri || true)
GIT_SHA=$(md instance/attributes/git-sha || echo unknown)
BUCKET=$(md instance/attributes/gcs-bucket || true)
REPORT_PREFIX=$(md instance/attributes/report-prefix || echo nightly/reports)
WEIGHTS_URI=$(md instance/attributes/weights-uri || true)
DO_EMAIL=$(md instance/attributes/email || echo true)
```

```
SMTP_SECRET=$(md instance/attributes/smtp-secret || true)
SMTP_USER=$(md instance/attributes/smtp-user || true)
EMAIL_TO=$(md instance/attributes/email-to || echo avi.dullu@gmail.com)
DATE=$(date -u +%F)

# ALWAYS delete the VM on exit (success OR failure) ? never leave a billing GPU VM behind.
cleanup() {
  echo "===== self-delete $NAME ($ZONE) ====="
  gcloud compute instances delete "$NAME" --zone="$ZONE" --quiet || \
    echo "WARN: self-delete failed ? DELETE MANUALLY: gcloud compute instances delete $NAME --zone=$ZONE -q"
}
trap cleanup EXIT

fail_email() {  # best-effort failure alert (the report email won't have fired on an early crash)
  echo "RUN FAILED ? attempting a failure alert"
  if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]; then
    cd ~/rally 2>/dev/null && . .venv/bin/activate 2>/dev/null && \
    NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER" NIGHTLY_EMAIL_TO="$EMAIL_TO" \
    python - <<PY || true
from scripts.nightly_email import send_report
send_report("[KhelSutra nightly] ERROR ? engine regression VM run failed ($DATE)",
            "The nightly engine-regression run failed on the VM before producing a report.\n"
            "See gs://$BUCKET/$REPORT_PREFIX/$DATE/ and the VM serial log.\nGIT_SHA=$GIT_SHA", to="$EMAIL_TO")
PY
  fi
}
trap 'fail_email' ERR

# 1) Observability (Ops Agent) ? same as create_gpu_vm.sh bakes in.
curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && \
  sudo bash add-google-cloud-ops-agent-repo.sh --also-install || echo "WARN: ops-agent install failed (non-fatal)"

# 2) Pull the repo @ HEAD (staged to GCS by launch.sh ? no GitHub auth needed; uses the worker SA).
mkdir -p ~/rally && cd ~/rally
gcloud storage cp "$REPO_TGZ" /tmp/rally.tgz
tar -xzf /tmp/rally.tgz -C ~/rally
echo "$GIT_SHA" > ~/rally/GIT_SHA

# 3) Build the env + native WASB (provider-agnostic DO scripts, reused on GCP per deploy/gcp/README §4).
sudo bash deploy/digitalocean/mount_scratch.sh 2>/dev/null || true
export TMPDIR=/scratch 2>/dev/null || true
./deploy/digitalocean/bootstrap.sh --gpu
. .venv/bin/activate
pip install -e '.[storage-gcs]'
bash deploy/digitalocean/gpu_setup.sh || echo "WARN: gpu_setup (WASB env) returned nonzero ? continuing"

# 4) Stage the WASB checkpoint (gpu_setup does NOT fetch weights ? README §4).
if [ -n "$WEIGHTS_URI" ]; then
  mkdir -p ~/models/WASB-SBDT/pretrained_weights
  gcloud storage cp "$WEIGHTS_URI" ~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
  export WASB_WEIGHTS=~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
fi

# 5) Run the regression on the gs:// golden clips in the committed manifest. Cost = VM
```

```
uptime so far.
UPTIME=$(( $(date +%s) - BOOT_EPOCH ))
EMAIL_FLAG=""; [ "$DO_EMAIL" = "true" ] && EMAIL_FLAG="--email --email-to $EMAIL_TO"
set +e
NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER" NIGHTLY_EMAIL_TO="$EMAIL_TO" \
  python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG
RUN_RC=$?
set -e
echo "runner exit code: $RUN_RC  (0 pass / 1 regression / 4 stale)"

# 6) Upload the dated report to GCS for the record (alongside the email).
if [ -n "$BUCKET" ] && [ -d output/nightly_reelcount ]; then
  gcloud storage cp -r "output/nightly_reelcount/$DATE."* "gs://$BUCKET/$REPORT_PREFIX/$DATE/" || \
    gcloud storage cp -r output/nightly_reelcount "gs://$BUCKET/$REPORT_PREFIX/$DATE/" || true
fi

echo "===== nightly-regression done rc=$RUN_RC $(date -u +%FT%TZ) ====="
trap - ERR   # disable the failure-email trap; the EXIT (self-delete) trap stays
# the EXIT trap deletes the VM

```

6. user (2026-06-22T09:49:30.024Z)

```
#!/usr/bin/env bash
# Launch ONE nightly engine-regression run on an ephemeral GCP GPU VM, then return. The VM runs
# run_on_vm.sh as its startup-script (regression ? email ? SELF-DELETE), so this just stages the repo
# and kicks the VM off. Driver-agnostic: call it from Cloud Scheduler?Cloud Run job, a cron on any
# always-on box, the owner's machine, or a Claude routine. Needs gcloud auth + the repo checkout.
#
#   bash deploy/gcp/nightly_regression/launch.sh                 # L4, stages HEAD, kicks the VM
#   RALLY_GPU=t4 RALLY_REF=$(git rev-parse HEAD) bash deploy/gcp/nightly_regression/launch.sh
#
# ? Run this MANUALLY once as a dry-run (watch the serial log) before wiring a scheduler ? the VM-side
# flow is NOT covered by CI. Tail: gcloud compute instances get-serial-port-output <name> --zone=<z>.
set -uo pipefail

PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
BUCKET="${RALLY_BUCKET:-khelsutra-rally-corpus}"
SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
GPU="${RALLY_GPU:-l4}"
NAME="${RALLY_VM_NAME:-rally-nightly-$(date -u +%Y%m%d)}"
REF="${RALLY_REF:-HEAD}"
WEIGHTS_URI="${RALLY_WEIGHTS_URI:-gs://$BUCKET/weights/wasb_badminton_best.pth.tar}"
REPORT_PREFIX="${RALLY_REPORT_PREFIX:-nightly/reports}"
DO_EMAIL="${RALLY_EMAIL:-true}"
SMTP_SECRET="${NIGHTLY_SMTP_SECRET:-projects/$PROJECT/secrets/nightly-smtp-pass/versions/latest}"
SMTP_USER="${NIGHTLY_SMTP_USER:-}"
EMAIL_TO="${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
STARTUP="$HERE/run_on_vm.sh"
export CLOUDSDK_CORE_DISABLE_PROMPTS=1

ZONES="${RALLY_GCP_ZONES:-asia-south1-a asia-south1-b asia-south1-c asia-southeast1-a
```

```
asia-southeast1-b asia-southeast1-c}"
if [ "$GPU" = "l4" ]; then MACHINE="g2-standard-4"; ACCEL=(); else MACHINE="n1-standard-8"; ACCEL=(--accelerator=type=nvidia-tesla-t4,count=1); fi

# 1) Stage the repo @ REF to GCS (no GitHub auth on the VM; the worker SA can read the bucket).
SHA="$(git rev-parse "$REF")"
TGZ_URI="gs://$BUCKET/nightly/repo/${SHA}.tgz"
echo "== staging repo $SHA -> $TGZ_URI =="
TMP_TGZ="$(mktemp --suffix=.tgz)"
git archive --format=tar.gz -o "$TMP_TGZ" "$REF"
gcloud storage cp "$TMP_TGZ" "$TGZ_URI" --project="$PROJECT"
rm -f "$TMP_TGZ"

# 2) Zone-sweep create the VM (mirrors create_gpu_vm.sh) with run_on_vm.sh + the config metadata.
ERR="$(mktemp)"
for Z in $ZONES; do
  echo "== creating $NAME ($GPU) in $Z =="
  if gcloud compute instances create "$NAME" --project="$PROJECT" --zone="$Z" \
       --machine-type="$MACHINE" "${ACCEL[@]}" \
       --maintenance-policy=TERMINATE --provisioning-model=STANDARD \
       --image-family=common-cu129-ubuntu-2204-nvidia-580 --image-project=deeplearning-platform-release \
       --boot-disk-size=120GB --boot-disk-type=pd-balanced \
       --service-account="$SA" --scopes=cloud-platform \
       --metadata-from-file=startup-script="$STARTUP" \
       --metadata=repo-tgz-uri="$TGZ_URI",git-sha="$SHA",gcs-bucket="$BUCKET",report-prefix="$REPORT_PREFIX",weights-uri="$WEIGHTS_URI",email="$DO_EMAIL",smtp-secret="$SMTP_SECRET",smtp-user="$SMTP_USER",email-to="$EMAIL_TO" \
       2>"$ERR"; then
    echo "? launched $NAME in $Z ? it will run the regression, email $EMAIL_TO, and self-delete."
    echo "   serial log: gcloud compute instances get-serial-port-output $NAME --zone=$Z --project=$PROJECT"
    echo "   if it doesn't self-delete: gcloud compute instances delete $NAME --zone=$Z -q"
    rm -f "$ERR"; exit 0
  fi
  if grep -qiE "STOCKOUT|does not have enough" "$ERR"; then echo "  (no $GPU capacity in $Z ? next zone)"; continue; fi
  echo "  create failed:"; tail -4 "$ERR"; rm -f "$ERR"; exit 1
done
echo "? no $GPU capacity in any swept zone ($ZONES). Retry later or RALLY_GPU=t4."
rm -f "$ERR"; exit 1

```

7. user (2026-06-22T09:49:30.940Z)

```
#!/usr/bin/env bash
# Wire the nightly engine-regression as a CLOUD-NATIVE, scale-to-zero job:
#   Cloud Scheduler (cron)  ?  Cloud Run JOB (the launcher image)  ?  launch.sh
#     ?  ephemeral GPU VM (regression + email + SELF-DELETE).
# The only always-on cost is $0 (Cloud Run jobs + Scheduler are scale-to-zero; the GPU VM is ephemeral).
#
# ? OWNER-RUN, ONCE, after a successful manual `bash deploy/gcp/nightly_regression/launch.sh` dry-run.
# NOT verified in CI (no GCP/GPU here). Prereqs: GPU quota (README §2), the golden clips + weights in
# gs://<bucket>, and the SMTP secret (see README "Enable").
#
#   bash deploy/gcp/nightly_regression/register_scheduler.sh           # build + deploy + schedule (03:00 IST)
#   RALLY_CRON='0 3 * * *' RALLY_TZ='Asia/Kolkata' bash .../register_scheduler.sh
```

```
set -euo pipefail

PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
REGION="${RALLY_RUN_REGION:-asia-south1}"
JOB="${RALLY_RUN_JOB:-nightly-regression-launcher}"
SCHED="${RALLY_SCHED_NAME:-nightly-regression}"
CRON="${RALLY_CRON:-0 3 * * *}"          # 03:00 daily
TZ="${RALLY_TZ:-Asia/Kolkata}"
SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
IMAGE="${RALLY_LAUNCHER_IMAGE:-${REGION}-docker.pkg.dev/${PROJECT}/rally/nightly-launcher:latest}"
REPO_ROOT="$(git rev-parse --show-toplevel)"
export CLOUDSDK_CORE_DISABLE_PROMPTS=1

echo "== enable APIs (run, scheduler, build, artifact registry) =="
gcloud services enable run.googleapis.com cloudscheduler.googleapis.com \
  cloudbuild.googleapis.com artifactregistry.googleapis.com --project="$PROJECT"

echo "== ensure the Artifact Registry repo 'rally' exists =="
gcloud artifacts repositories describe rally --location="$REGION" --project="$PROJECT" >/dev/null 2>&1 || \
  gcloud artifacts repositories create rally --repository-format=docker --location="$REGION" --project="$PROJECT"

echo "== build + push the launcher image from the repo root =="
( cd "$REPO_ROOT" && gcloud builds submit --project="$PROJECT" --tag "$IMAGE" \
    --gcs-source-staging-dir="gs://${RALLY_BUILD_BUCKET:-khelsutra-rally-corpus}/cloudbuild" \
    -f deploy/gcp/nightly_regression/Dockerfile.launcher . )

echo "== create/update the Cloud Run JOB (launcher needs compute-admin to create the GPU VM) =="
gcloud run jobs deploy "$JOB" --project="$PROJECT" --region="$REGION" \
  --image="$IMAGE" --service-account="$SA" --max-retries=0 --task-timeout=900s \
  --set-env-vars="RALLY_GPU=${RALLY_GPU:-l4},RALLY_EMAIL=true,NIGHTLY_EMAIL_TO=${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"

echo "== schedule it (Cloud Scheduler ? Run job admin API) =="
RUN_URI="https://${REGION}-run.googleapis.com/apis/run.googleapis.com/v1/namespaces/${PROJECT}/jobs/${JOB}:run"
gcloud scheduler jobs describe "$SCHED" --location="$REGION" --project="$PROJECT" >/dev/null 2>&1 && \
  gcloud scheduler jobs delete "$SCHED" --location="$REGION" --project="$PROJECT" --quiet || true
gcloud scheduler jobs create http "$SCHED" --project="$PROJECT" --location="$REGION" \
  --schedule="$CRON" --time-zone="$TZ" \
  --uri="$RUN_URI" --http-method=POST \
  --oauth-service-account-email="$SA"

cat <<EOF

? Scheduled '$SCHED' ($CRON $TZ) ? Cloud Run job '$JOB' ? launch.sh ? ephemeral GPU VM.
   IAM the launcher SA needs (grant once if missing):
     roles/compute.instanceAdmin.v1  (create/delete the GPU VM)
     roles/iam.serviceAccountUser    (attach $SA to the VM)
     roles/run.invoker + roles/cloudscheduler.jobRunner (Scheduler ? Run)
   Trigger now to test:  gcloud run jobs execute $JOB --region=$REGION --project=$PROJECT
   Remove:               gcloud scheduler jobs delete $SCHED --location=$REGION; gcloud run jobs delete $JOB --region=$REGION
EOF

```

8. user (2026-06-22T09:49:31.864Z)

```
# Tiny launcher image for the nightly engine-regression Cloud Run JOB. It is NOT the GPU workload ?
# it only stages the repo + creates the ephemeral GPU VM (which self-deletes). Scale-to-zero: the job
# runs for a few seconds nightly, then exits. Base = google/cloud-sdk (gcloud + git preinstalled).
FROM gcr.io/google.com/cloudsdktool/cloud-sdk:slim
WORKDIR /app
# The repo is the build context (run `gcloud builds submit` from the repo root, see register_scheduler.sh).
COPY . /app
ENTRYPOINT ["bash", "deploy/gcp/nightly_regression/launch.sh"]

```

9. user (2026-06-22T09:49:32.258Z)

```
{
  "_doc": "Nightly E2E engine reel-count manifest (scripts/nightly_reelcount.py). The configured+checkpointed pipeline runs on each video on a GCP GPU VM; the reel count + quality metrics are asserted vs eval_baselines/reelcount_baseline.json. ADD A VIDEO = append one row + re-freeze: `python scripts/nightly_reelcount.py --update-baseline` (commit the updated baseline). URIs may be gs:// (recommended, co-located with the VM), gdrive://, or local paths. labels_uri (optional) is a golden-features JSON (gts key) or a start,end CSV ? present => quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the example rows below to your actual short (~5-6 min) OWNED, minors-free clips before enabling.",
  "segmenter": "wasb_hybrid",
  "detector_impl": "native",
  "config_overrides": {
    "indexing.default_segmenter": "wasb_hybrid",
    "indexing.detector_impl": "native"
  },
  "machine_usd_per_hr": 0.70,
  "fx_inr_per_usd": 83.0,
  "videos": [
    {
      "name": "GX030094",
      "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX030094.mp4",
      "labels_uri": "gs://khelsutra-rally-corpus/nightly/labels/GX030094_golden_features.json"
    },
    {
      "name": "Badminton_BXH_2",
      "uri": "gs://khelsutra-rally-corpus/nightly/videos/Badminton_BXH_2.mp4",
      "labels_uri": "gs://khelsutra-rally-corpus/nightly/labels/Badminton_BXH_2_golden_features.json"
    },
    {
      "name": "GX010128",
      "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX010128.mp4",
      "labels_uri": "gs://khelsutra-rally-corpus/nightly/labels/GX010128_golden_features.json"
    }
  ]
}

```

10. assistant (2026-06-22T09:49:35.920Z)

Now let me examine the specific code section mentioned in the finding around lines 340-376:

11. user (2026-06-22T09:49:37.728Z)

```
340       def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float, float]]], float, Optional[str]]:
```

```
341             random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields are reproducible too
342             db = Database(db_path)
343             video_id = compute_video_id(local)
344             t0 = time.monotonic()
345             result = seg_cls(db, config).process_video(video_id, local)
346             wall = time.monotonic() - t0
347             if isinstance(result, Failure):
348                 return None, None, wall, getattr(result, "message", "segmenter failed")
349             segs = result.value if hasattr(result, "value") else result
350             intervals = _segments_to_intervals(segs)
351             return len(segs), intervals, wall, None
352
353         import tempfile
354         total_wall = 0.0
355         with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356             count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357             total_wall += wall
358             if err is not None:
359                 return {"name": name, "reel_count": None, "ok": False, "error": err,
360                         "quality": None, "wall_seconds": round(total_wall, 2)}
361             # re-run-once guard for the transient DB-drop flake
362             if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
363                 count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
364                 total_wall += wall2
365                 if err2 is None and count2 is not None:
366                     count, intervals = count2, intervals2  # the reproduced (or corrected) value
367
368             quality = None
369             if gts and intervals is not None:
370                 row = rally_seg_eval.score_intervals(intervals, gts)
371                 quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25", "tol_precision", "tol_recall")
372                            if k in row}
373
374         return {"name": name, "reel_count": count, "ok": True, "error": None,
375                 "quality": quality, "wall_seconds": round(total_wall, 2)}
376
377
378     def load_manifest(path: str) -> Dict[str, Any]:
379         with open(path, encoding="utf-8") as fh:
380             return json.load(fh)
381
382
383     def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
384                    rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
385         """Run every manifest video through the configured pipeline. Returns (run_results, total_wall_s)."""
386         from backend.config import load_config
387
388         config = load_config()
389         # apply manifest config overrides onto the typed config (dotted keys)
```

12. assistant (2026-06-22T09:49:54.651Z)

Perfect. Now I can verify the finding. Let me trace through the logic path carefully:

Analysis of the finding:

Looking at lines 347-375 in `run_one_video()`:

1. **Line 347-348**: If `isinstance(result, Failure)` is True, the function returns early with `(None, None, wall, error_message)` ? `count` is explicitly `None` here.

2. **Line 349-351**: If NOT a Failure, extract `segs` and call `_segments_to_intervals(segs)` at line 350. This returns `(len(segs), intervals, wall, None)` ? note `count` is `len(segs)`, which is an int >= 0.

3. **Line 356-357**: First run: `count, intervals, wall, err = _process(...)`. Now `count` could be:
   - `None` if `err is not None` (a Failure case)
   - An int `>= 0` if successful

4. **Line 358-360**: If `err is not None`, the function returns immediately with `reel_count=None`. This means we **never reach line 362**.

5. **Line 362**: This guard only executes if we **passed line 358's check**, meaning `err is None`. At this point, `count` is **guaranteed to be an int >= 0** (from the successful `_process` return at line 351).

6. **Line 362 comparison**: `count != baseline_count` is safe because `count` is an int.

The Finding's Assessment:

The finding claims: "If result is a Success with .value=None (a malformed segmenter), or result is a bare list=[], line 350 _segments_to_intervals([]) is called safely."

Let me check if `segs` could be `None`:
- Line 349: `segs = result.value if hasattr(result, "value") else result`
  - If `result.value` is `None`, then `segs = None`
  - Then line 350: `intervals = _segments_to_intervals(None)` would be called
  - Looking at `_segments_to_intervals` (lines 295-306), it iterates: `for s in segs:`
  - If `segs=None`, this would raise a **TypeError: 'NoneType' object is not iterable** at line 301

But this is actually NOT a bug in the nightly flow, because:
1. If `segs=None`, `len(segs)` at line 351 would **crash with TypeError** before returning
2. The function would **not return successfully** ? an exception would bubble up
3. The exception would be caught by the outer try-except in `main()` at line 487, returning exit code 2

However, there's a **subtle edge case** to reconsider:

Re-checking line 349 logic:
```python
segs = result.value if hasattr(result, "value") else result
```

If the segmenter returns a bare list `[]` or `None` directly (not wrapped), then:
- `segs = result` (the bare value)
- If `segs = None`, line 351 crashes on `len(segs)`
- If `segs = []`, line 351 returns `count=0` successfully

The actual REAL issue: If `result.value = None` (wrapped in a Success object), then:
- Line 349: `segs = result.value = None`
- Line 350: `intervals = _segments_to_intervals(None)` ? TypeError on line 301
- **This is NOT caught** ? it propagates as a crash, not a Failure

Let me verify if this is production-safe by checking if the segmenters can actually return `Success(None)`:

13. user (2026-06-22T09:49:57.377Z)

Found 4 files
tests\test_results.py
tests\test_base.py
backend\results.py
backend\pipeline\segmenters\base.py

14. user (2026-06-22T09:50:00.105Z)

```
from abc import ABC, abstractmethod
from typing import List, Dict, Any, Type, Optional
from backend.infrastructure.database import Database
from backend.results import Result  # new standardized contract (Result[Value] == Success[Value] | Failure)

class BasePlaySegmenter(ABC):
    def __init__(self, db: Database, config: Dict[str, Any]):
        self.db = db
        self.config = config

    @property
    @abstractmethod
    def name(self) -> str:
        """Unique identifier for this segmenter."""
        pass

    @property
    @abstractmethod
    def description(self) -> str:
        """Human-readable explanation of this segmenter."""
        pass

    @abstractmethod
    def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
        """
        Processes a video to detect play segments, populates SQLite database,
        and returns a Result.

        Use isinstance(result, Failure) (or result.is_success()) to distinguish
        "no segments found" from "something went wrong" (missing key, healthcheck
        failure, etc.). This is the start of the standardized error/result contract.
        """
        pass

class SegmenterRegistry:
    def __init__(self):
        self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}

    def register(self, segmenter_cls: Type[BasePlaySegmenter]):
        """Registers a new segmenter class under its name property."""
        try:
            temp_inst = segmenter_cls(None, {})
            name = temp_inst.name
        except Exception as e:
            raise ValueError(f"Failed to resolve name property from segmenter class {segmenter_cls.__name__}: {e}") from e
        self._segmenters[name] = segmenter_cls

    def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
        return self._segmenters.get(name)

    def list_available(self) -> Dict[str, str]:
        """Returns a dict of registered segmenter names and their descriptions."""
        res = {}
        for name, cls in self._segmenters.items():
            try:
                temp_inst = cls(None, {})
                desc = temp_inst.description
            except Exception:
                desc = "No description available."
            res[name] = desc
```

```
        return res

segmenter_registry = SegmenterRegistry()

```

15. user (2026-06-22T09:50:00.630Z)

```
"""Lightweight Result type for standardizing success vs. failure across the pipeline.

This is the start of the Error/Result Contract standardization (low-regret item #3).

The goal is to make it impossible to confuse "no data / no rallies" with "something actually failed"
(e.g. missing API key, WSL healthcheck failure, provider error, etc.).

We use simple dataclasses so callers can use isinstance checks or structural pattern matching.
This is intentionally small and incremental ? we start by applying it to the segmenter
and provider layers (the biggest sources of silent [] returns today).

The types are generic where it makes sense. Callers can still access .value on Success
or .message on Failure.

This module has no dependencies on the rest of the app.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Generic, TypeVar, List, Optional, Any, Tuple, Union

T = TypeVar("T")


@dataclass
class Success(Generic[T]):
    """Represents a successful outcome with an optional list of non-fatal warnings."""

    value: T
    warnings: List[str] = field(default_factory=list)

    def is_success(self) -> bool:
        return True

    def unwrap(self) -> T:
        """Return the value. Only safe to call if you already know it's a Success."""
        return self.value


@dataclass
class Failure:
    """Represents a failure with a human-readable message and optional details."""

    message: str
    details: Optional[dict[str, Any]] = None
    is_recoverable: bool = False  # e.g. missing API key (recoverable) vs internal crash

    def is_success(self) -> bool:
        return False

    def unwrap(self) -> Any:
        """Raise to make misuse obvious in code that assumes success."""
        raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")


```

```
# Convenient alias for use in type hints: Result[list[dict], Failure]
# Result[ValueType] means Success[ValueType] | Failure
# (the Failure "error" type is fixed for this contract)
Result = Union[Success[T], Failure]


def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
    """Split a segmenter/provider return into ``(value, failure)`` so a caller branches once.

    - ``Failure``            -> ``(None, failure)``
    - ``Success(value=v)``   -> ``(v, None)``
    - a bare value (legacy code that predates this contract and still returns a raw list)
      -> ``(value, None)``

    This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline; callers
    that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no len()``
    on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is heterogeneous
    (Success | Failure | legacy bare value)."""
    if isinstance(result, Failure):
        return None, result
    value = result.value if hasattr(result, "value") else result
    return value, None

```

16. assistant (2026-06-22T09:50:02.778Z)

Now let me check what the actual segmenters return:

17. user (2026-06-22T09:50:04.244Z)

```
backend\pipeline\segmenters\__init__.py
backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\_keyhint.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\yolo_hybrid.py
```

18. user (2026-06-22T09:50:07.741Z)

```
backend\pipeline\segmenters\motion.py:37:        return Success(value=segments_data)
```

19. assistant (2026-06-22T09:50:09.950Z)

Let me check a few segmenters to see what they return:

20. user (2026-06-22T09:50:10.938Z)

```
import cv2
import time
import random
from typing import List, Dict, Any, Tuple
import logging
from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
from backend.results import Success, Failure  # standardized contract

```

```
logger = logging.getLogger(__name__)

class MotionPlaySegmenter(BasePlaySegmenter):
    @property
    def name(self) -> str:
        return "motion"

    @property
    def description(self) -> str:
        return "Traditional CV pixel-motion heuristics tracking frame differences."

    def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
        """
        Processes a video to detect play segments using motion heuristics,
        populates SQLite with detected segments, and indexes detailed events.
        """
        if not player_pool:
            player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]

        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            logger.error(f"Segmenter could not open video: {video_path}")
            return Failure(message=f"Could not open video: {video_path}", is_recoverable=False)

        final_segments = self._detect_motion_segments(cap, max_frames)

        segments_data = self._populate_db_with_events(video_id, final_segments, player_pool)

        return Success(value=segments_data)

    def _detect_motion_segments(self, cap: Any, max_frames: int = None) -> List[Tuple[float, float]]:
        """Run the pixel-motion heuristic over ``cap`` and return the detected
        (start, end) play segments.

        Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
        thresholds it into raw segments, then merges dead-time gaps and filters
        out segments shorter than ``min_segment_duration``. ``cap`` is released
        before returning. This is the detection half of ``process_video`` ? it
        performs no DB writes and no fabrication.
        """
        fps = cap.get(cv2.CAP_PROP_FPS)
        total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
```

21. user (2026-06-22T09:50:11.674Z)

```
"""WASB-based hybrid play segmenter.

Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
badminton weights and runs today, so this is the live shuttle-motion substrate.

For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
min_window_duration) are what get tuned against golden rally labels.

The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
WASB runner and identity.
"""
from typing import Any, Dict, Tuple

```

```
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
from backend.pipeline.detectors.base import DetectorRunner
from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig


class WasbHybridSegmenter(TrajectoryHybridSegmenter):
    family = "wasb"
    display_label = "WASB"

    @property
    def name(self) -> str:
        return "wasb_hybrid"

    @property
    def description(self) -> str:
        return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained badminton "
                "weights) for precise candidate rallies + AI provider for semantic boundaries.")

    def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        cfg = WasbConfig.from_indexing_cfg(idx_cfg)
        return WasbRunner(cfg), {"weights_path": cfg.weights_path}

    def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        # Lazy import: the native runner pulls no heavy deps, but keep parity with the stub
        # seam (imported only when detector_impl='native' is actually selected).
        from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner, NativeWasbConfig
        cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
        return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
                                       "detector_impl": "native", "device": cfg.device}


segmenter_registry.register(WasbHybridSegmenter)

```

22. user (2026-06-22T09:50:14.788Z)

```
"""Shared engine for trajectory-substrate hybrid segmenters.

tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
now the single home for the whole pipeline shape:

    healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
    -> P3 AI-provider handoff over padded clips -> P4 DB population/linking

A concrete subclass supplies only its identity and runner wiring:
- ``family``        ? config section under ``indexing.*``, output subdir, and
                      the ``<family>-hybrid-<model>`` detector tag in metadata.
- ``display_label`` ? human label for log lines ("WASB", "TrackNet").
- ``name`` / ``description`` properties (the registry contract).
- ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
                      detection_params extras such as weights_path).

This base is deliberately NOT registered; only concrete subclasses register.
"""
import os
import time
import logging
```

```
import concurrent.futures
from typing import Any, Dict, List, Optional, Tuple

import cv2

from backend.pipeline.segmenters.base import BasePlaySegmenter
from backend.utils.video import get_video_duration, extract_video_chunk
from backend.utils.paths import output_base
from backend.sports import sport_registry
from backend.providers import provider_registry
from backend.pipeline.detectors.base import DetectorRunner

logger = logging.getLogger(__name__)


def _fmt_ts(seconds: float) -> str:
    seconds = max(0, int(seconds))
    h, rem = divmod(seconds, 3600)
    m, s = divmod(rem, 60)
    return f"{h:02d}:{m:02d}:{s:02d}"


class TrajectoryHybridSegmenter(BasePlaySegmenter):
    """Trajectory-detector hybrid: precise candidate rallies + AI semantic boundaries."""

    #: config section under `indexing` (velocity_threshold lives there), the
    #: output subdir for trajectories, and the metadata detector-tag prefix.
    family: str = ""
    #: human-readable label used in log lines.
    display_label: str = ""

    def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        """Return (runner, detector-specific detection_params extras) for the default (WSL) path."""
        raise NotImplementedError

    def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        """Return (runner, extras) for the Linux-native (no-WSL) path. Only families with a
        native runner override this; the base refuses with a clear message (M3.5: WASB only)."""
        raise NotImplementedError(
            f"detector_impl='native' is not supported for the "
            f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB has a "
            f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")

    def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        """Resolve the detector runner, honouring the CPU-stub and Linux-native overrides.

        ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or ``indexing.detector_impl``:
        ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole pipeline
        is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"`` swaps
        in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
        ``_build_native_runner``. Anything else (``"auto"``) delegates to ``_build_runner``
        (the WSL runner) so the default production path is unchanged
```

```
        (``docs/DECOUPLED_COMPUTE/``).
        """
        impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or "auto").lower()
        if impl == "stub":
            from backend.pipeline.detectors.stub_runner import StubDetectorRunner, StubConfig
            logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
                        self.display_label or "trajectory")
            return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)), {"detector_impl": "stub"}
        if impl in ("native", "native_wasb", "wasb_native"):
            logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
                        self.display_label or "trajectory")
            return self._build_native_runner(idx_cfg)
        return self._build_runner(idx_cfg)

    @staticmethod
    def _probe_fps_width(video_path: str) -> tuple:
        """Return (fps, frame_width) for a video, with sensible fallbacks."""
        cap = cv2.VideoCapture(video_path)
        fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
        width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
        cap.release()
        return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)

    @staticmethod
    def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
        """Provider-reported exchange count, else a duration-based estimate.

        One canonical implementation ? the twins had drifted (wasb relied on
        ``int(None)`` raising into the fallback; same result, by accident).
        """
        shot_count = segment.get("shuttle_exchange_count")
        if shot_count is None:
            return max(3, int(duration / 0.8))
        try:
            return int(shot_count)
        except (ValueError, TypeError):
            return max(3, int(duration / 0.8))

    # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged) -------
    def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                frame_width: float, video_path: str,
                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
        """Stats dict persisted into ``candidate_segments.stats`` for one window. The BASE
        returns exactly the 4 motion keys, so the trajectory twins persist byte-identical
        rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
        features). ``points`` are the whole-video trajectory points (TrajectoryPoint)."""
        return {
            "peak_velocity": cw["peak_velocity"],
            "mean_velocity": cw["mean_velocity"],
            "active_frame_count": cw["active_frame_count"],
            "track_id_count": cw["track_id_count"],
        }

    def _select_for_handoff(self, windows: List[Dict[str, Any]],
                            window_stats: List[Dict[str, Any]],
                            idx_cfg: Dict[str, Any]) -> List[int]:
        """Indices of the candidate windows that proceed to the AI handoff (and thus may
        become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
```

```
        ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
        Persisted candidates are NOT affected ? they remain the full superset for offline
        re-scoring."""
        return list(range(len(windows)))

    def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
                      player_pool: Optional[List[str]] = None,
                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
        # override-ignore: the ABC declares the Result contract (Success|Failure)
        # but every live segmenter still returns a bare list ? the documented
        # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
        label = self.display_label
        # --- Sport profile + AI provider wiring (identical across segmenters) ---
        sport_name = self.config.get("sport", "badminton")
        try:
            sport_profile = sport_registry.get(sport_name)
        except KeyError as e:
            logger.error(str(e))
```

23. user (2026-06-22T09:50:18.356Z)

```
    def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
                      player_pool: Optional[List[str]] = None,
                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
        # override-ignore: the ABC declares the Result contract (Success|Failure)
        # but every live segmenter still returns a bare list ? the documented
        # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
        label = self.display_label
        # --- Sport profile + AI provider wiring (identical across segmenters) ---
        sport_name = self.config.get("sport", "badminton")
        try:
            sport_profile = sport_registry.get(sport_name)
        except KeyError as e:
            logger.error(str(e))
            return []

        idx_cfg = self.config.get("indexing", {})
        # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become the
        # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key is
        # needed and nothing is uploaded. Used to run the local detector standalone (e.g. to
        # measure it against the Gemini path, or to avoid the cloud entirely). With
        # FusionSegmenter the windows are still Gate-A/B-filtered via `_select_for_handoff`.
        skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
        provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))
        if skip_ai:
            model_name = "local-noai"
            logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
                        "be emitted as rallies; no %s handoff, no API key needed.",
                        label, provider_name)
        else:
            model_name = self.config.get("ai_model", idx_cfg.get("ai_model", "gemini-2.5-flash"))
            api_key_env_var = f"{provider_name.upper()}_API_KEY"
            api_key = os.environ.get(api_key_env_var)
            if not api_key:
                from backend.pipeline.segmenters._keyhint import api_key_hint
                logger.error(
                    f"{api_key_env_var} environment variable is not set! "
```

```
                    f"To run the {provider_name} handoff, set your API key. "
                    + api_key_hint(api_key_env_var)
                )
                return []
            try:
                provider = provider_registry.get(provider_name, api_key=api_key, model_name=model_name)
            except KeyError as e:
                logger.error(str(e))
                return []

        video_duration = get_video_duration(video_path)
        if video_duration <= 0:
            logger.error("Invalid video duration.")
            return []
        fps, frame_width = self._probe_fps_width(video_path)

        # --- Runner config + windowing thresholds ---
        # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
        # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
        # it delegates to the subclass _build_runner (the real WSL/native runner).
        try:
            runner, runner_params = self._resolve_runner(idx_cfg)
        except NotImplementedError as e:
            # e.g. detector_impl="native" for a family without a native runner (tracknet, or
            # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of crashing.
            logger.error("%s: %s", label, e)
            return []

        velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
        merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
        min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
        # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See IndexingConfig.
        max_window_duration = idx_cfg.get("max_window_duration", 0.0)
        window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
        window_hard_cap = idx_cfg.get("window_hard_cap", False)
        window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
        inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
        inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
        window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
        window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
        handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
        ai_max_workers = idx_cfg.get("ai_max_workers", 5)
        log_candidates = idx_cfg.get("log_yolo_candidates", True)
        store_candidates = idx_cfg.get("store_yolo_candidates", True)

        detection_params = {
            "detector": self.name,
            "velocity_thresh": velocity_thresh,
            "merge_gap": merge_gap,
            "min_action_window_duration": min_window_duration,
            **runner_params,
        }

        # --- Phase 1: env healthcheck (fail fast with a clear message) ---
        ok, msg = runner.healthcheck()
        if not ok:
            logger.error("%s detector environment not ready: %s", label, msg)
            return []
        logger.info("%s detector env OK: %s", label, msg)

        # --- Phase 2: trajectory -> candidate rally windows ---
```

```
        # Trajectory detectors process the whole video in one pass (they carry
        # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
        t_start = time.time()
        out_dir = os.path.join(output_base(self.config), self.family)
        csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
        if not csv_path:
            logger.error("%s produced no trajectory; aborting.", label)
            return []

        points = runner.parse_trajectory_csv(csv_path)
        logger.info("Parsed %d trajectory points (%d visible).",
                    len(points), sum(1 for p in points if p.visible))

        all_candidate_windows = runner.trajectory_to_action_windows(
            points, fps=fps, frame_width=frame_width, chunk_start=0.0,
            velocity_thresh=velocity_thresh, merge_gap=merge_gap,
            min_window_duration=min_window_duration, chunk_index=0,
            max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
            window_hard_cap=window_hard_cap,
            window_inactivity_end=window_inactivity_end,
            inactivity_rally_thresh=inactivity_rally_thresh,
            inactivity_min_density=inactivity_min_density,
            window_blob_fallback=window_blob_fallback,
            window_blob_factor=window_blob_factor,
        )
        logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                    label, time.time() - t_start, len(all_candidate_windows))

        if log_candidates:
            for cw in all_candidate_windows:
                logger.info(f"[{label} rally]
{_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                                f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                                f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

        # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
        # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
        # persistence and the handoff gate below.
        window_stats = [
            self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
            for cw in all_candidate_windows
        ]

        # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
        candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
        if store_candidates:
            for i, cw in enumerate(all_candidate_windows):
                candidate_ids[i] = self.db.add_candidate_segment(
                    video_id, detector=self.name,
                    start_time=cw["start"], end_time=cw["end"],
                    chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
                    stats=window_stats[i], detection_params=detection_params,
                )

```

24. assistant (2026-06-22T09:50:20.779Z)

Perfect. Now I can see the critical detail. Let me check the return statements:

25. user (2026-06-22T09:50:23.193Z)

```
151:            return []
177:                return []
182:                return []
187:            return []
200:            return []
231:            return []
242:            return []
289:            return []
325:                    return []
```

26. assistant (2026-06-22T09:50:25.347Z)

Now let me check the actual return of segments from trajectory_hybrid:

27. user (2026-06-22T09:50:26.473Z)

```
320                     w_start = max(0, window["start"] - handoff_padding)
321                     w_end = min(video_duration, window["end"] + handoff_padding)
322                     w_dur = w_end - w_start
323                     w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
324                     if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
325                         return []
326                     prompt = sport_profile.get_prompt(chunk_duration=w_dur)
327                     segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
328                                                          label=f"Handoff {wi+1}")
329                     valid = []
330                     for seg in segments:
331                         try:
332                             seg["start_time"] = float(seg["start_time"]) + w_start
333                             seg["end_time"] = float(seg["end_time"]) + w_start
334                             seg["_candidate_id"] = candidate_ids[wi]
335                             valid.append(seg)
336                         except (ValueError, TypeError, KeyError) as e:
337                             logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
338                     try:
339                         os.remove(w_path)
340                     except OSError:
341                         pass
342                     return valid
343
344                 with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
345                     futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
346                                for wi in handoff_indices]
347                     for future in futures:
348                         ai_segments.extend(future.result())
349                 try:
350                     os.rmdir(handoff_dir)
351                 except OSError:
352                     pass
353                 logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
354
355             # --- Phase 4: DB population + candidate linking ---
356             segments_data = []
357             for segment in ai_segments:
358                 try:
359                     start = float(segment["start_time"])
360                     end = float(segment["end_time"])
```

```
361                     dur = end - start
362                     metadata = {
363                         "shot_count": self._shot_count_from(segment, dur),
364                         "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
365                         "detector": f"{self.family}-hybrid-{model_name}",
366                     }
367                     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
368                     if segment_id:
369                         candidate_id = segment.get("_candidate_id")
```

28. user (2026-06-22T09:50:30.465Z)

```
355             # --- Phase 4: DB population + candidate linking ---
356             segments_data = []
357             for segment in ai_segments:
358                 try:
359                     start = float(segment["start_time"])
360                     end = float(segment["end_time"])
361                     dur = end - start
362                     metadata = {
363                         "shot_count": self._shot_count_from(segment, dur),
364                         "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
365                         "detector": f"{self.family}-hybrid-{model_name}",
366                     }
367                     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
368                     if segment_id:
369                         candidate_id = segment.get("_candidate_id")
370                         if candidate_id is not None:
371                             self.db.link_candidate_to_segment(candidate_id, segment_id)
372                         segments_data.append({
373                             "id": segment_id, "start_time": start, "end_time": end,
374                             "duration": dur, "metadata": metadata,
375                         })
376                 except Exception as e:
377                     logger.warning("Skipping invalid segment struct: %s", e)
378
379             return segments_data
380
```

29. assistant (2026-06-22T09:50:33.670Z)

Now I have all the information needed. Let me verify the critical edge case around line 367:

The issue is at **line 367**: `segment_id = self.db.add_play_segment(...)`. The function can return `None` if there's a transient database insertion error. Let me check if this is documented:

30. user (2026-06-22T09:50:35.588Z)

```
backend\infrastructure\database.py:459:    def add_play_segment(self, video_id: str, start_time:
float, end_time: float, server: Optional[str] = None, receiver: Optional[str] = None, metadata:
Optional[Dict[str, Any]] = None) -> Optional[int]:
```

31. user (2026-06-22T09:50:38.420Z)

```
459         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
460             try:
461                 with self._connect() as conn:
```

```
                cursor = conn.cursor()
                cursor.execute(
                    "INSERT INTO play_segments (video_id, start_time, end_time, duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
                    (video_id, start_time, end_time, end_time - start_time, server, receiver, json.dumps(metadata or {}))
                )
                conn.commit()
                return cursor.lastrowid
        except Exception as e:
            logger.error(f"Failed to add play segment: {e}")
            return None

    def add_event(self, segment_id: int, timestamp: float, event_type: str, player: Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
        return self._execute_write(
            "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES (?, ?, ?, ?, ?)",
            (segment_id, timestamp, event_type, player, json.dumps(details or {})),
            "Failed to add event",
        )

    def add_candidate_segment(self, video_id: str, detector: str, start_time: float, end_time: float,
                              chunk_index: Optional[int] = None, confidence: Optional[float] = None,
                              stats: Optional[Dict[str, Any]] = None,
                              detection_params: Optional[Dict[str, Any]] = None) -> Optional[int]:
        """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
        detector-specific dicts stored as JSON. Returns the new candidate id, or None."""
        try:
            with self._connect() as conn:
                cursor = conn.cursor()
                cursor.execute(
                    """INSERT INTO candidate_segments
                       (video_id, detector, chunk_index, start_time, end_time, duration,
                        confidence, stats, detection_params)
                       VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                    (video_id, detector, chunk_index, start_time, end_time, end_time - start_time,
                     confidence, json.dumps(stats or {}), json.dumps(detection_params or {}))
                )
                conn.commit()
                return cursor.lastrowid
        except Exception as e:
            logger.error(f"Failed to add candidate segment: {e}")
            return None

    def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
        """Record that a candidate detection was confirmed as a given play_segment."""
        return self._execute_write(
            "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
            (segment_id, candidate_id),
            f"Failed to link candidate {candidate_id} to segment {segment_id}",
```

32. assistant (2026-06-22T09:50:57.212Z)

Excellent! Now I have complete understanding. Let me synthesize the findings:

Key Code Paths:

1. **`trajectory_hybrid.py:367`**: `segment_id = self.db.add_play_segment(...)` can return

`None` on transient DB error (lines 469-471).

2. **`trajectory_hybrid.py:368-375`**: The code checks `if segment_id:` and only appends to `segments_data` if `segment_id` is truthy. If `segment_id is None`, the segment is silently skipped ? `segments_data` won't have this entry.

3. **`trajectory_hybrid.py:379`**: Returns `segments_data` (a list, possibly with fewer entries than `ai_segments`).

4. **`nightly_reelcount.py:345`**: `result = seg_cls(...).process_video(...)` returns a bare list (NOT wrapped in `Success()`), per the comment at line 141-143 ("every live segmenter still returns a bare list").

5. **`nightly_reelcount.py:349`**: `segs = result.value if hasattr(result, "value") else result` ? since `result` is a bare list, `segs = result` (the list itself).

6. **`nightly_reelcount.py:350`**: `intervals = _segments_to_intervals(segs)` is called safely with a list.

7. **`nightly_reelcount.py:351`**: `return len(segs), intervals, wall, None` ? `count = len(segs)` which is safe (the integer count of actual DB-inserted segments).

The REAL Issue (not the finding's claim):

The finding claims there's a subtle None-handling bug. However, the actual flow is:
- If `add_play_segment()` returns `None`, the segment is **skipped** (not added to `segments_data`)
- So `segments_data` only contains successfully-inserted segments
- `len(segs)` is thus the count of **actually persisted** play segments
- The rerun guard at line 362 uses this count ? it's safe

BUT THERE IS A REAL BUG that the finding doesn't capture:

At **`trajectory_hybrid.py:368-375`**, if `segment_id is None`, the segment is **silently dropped** from `segments_data`. This means:
- `ai_segments` may have N segments confirmed by the AI
- But only M < N are successfully persisted to the DB
- `segments_data` counts as M
- **The reel count in the nightly report is M, not N**
- If this happens intermittently (transient DB error), the nightly check's rerun guard **may not catch it** because:
  - First run: `count = M` (transient DB drop)
  - Baseline: `baseline_count = N` (from a clean run)
  - Comparison: `count (M) != baseline_count (N)` ? triggers rerun
  - Second run: `count2 = M` or `count2 = N` (depends if the transient error reproduces)
  - If the second run is clean (`count2 = N`), the rerun guard corrects it ?
  - If the second run also has the transient error (`count2 = M`), the guard doesn't help

HOWEVER, the code at `nightly_reelcount.py:362-366` **correctly handles this**:
- It reruns ONCE on mismatch
- It takes the reproduced/corrected value
- This is exactly the "re-run-once guard for the transient DB-drop flake" mentioned in the docstring

So the current code is **defensively correct**. The finding's "subtle but correct logic path" assessment is accurate.

Let me verify there are NO actual bugs:

33. user (2026-06-22T09:51:00.021Z)

segs=2, intervals=2
Test: _segments_to_intervals logic is safe

34. assistant (2026-06-22T09:51:02.035Z)

Now let me verify one more critical issue: what if `segs` is actually `None` at line 349?

35. user (2026-06-22T09:51:05.398Z)

segs = None, type = <class 'NoneType'>
ERROR: 'NoneType' object is not iterable

36. assistant (2026-06-22T09:51:07.858Z)

FOUND IT! This is a **REAL defect**. Let me verify that segmenters can actually return `Success(None)`:

37. user (2026-06-22T09:51:09.855Z)

No matches found

38. assistant (2026-06-22T09:51:12.267Z)

Good. Let me check if any segmenter returns `Success(value=[])` or `None`:

39. user (2026-06-22T09:51:13.332Z)

backend/pipeline/segmenters/motion.py

40. user (2026-06-22T09:51:15.742Z)

```
import cv2
import time
import random
from typing import List, Dict, Any, Tuple
import logging
from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
from backend.results import Success, Failure  # standardized contract

logger = logging.getLogger(__name__)

class MotionPlaySegmenter(BasePlaySegmenter):
    @property
    def name(self) -> str:
        return "motion"

    @property
    def description(self) -> str:
        return "Traditional CV pixel-motion heuristics tracking frame differences."

    def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
        """
        Processes a video to detect play segments using motion heuristics,
        populates SQLite with detected segments, and indexes detailed events.
        """
        if not player_pool:
            player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]

        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            logger.error(f"Segmenter could not open video: {video_path}")
            return Failure(message=f"Could not open video: {video_path}", is_recoverable=False)

        final_segments = self._detect_motion_segments(cap, max_frames)

        segments_data = self._populate_db_with_events(video_id, final_segments, player_pool)
```

```python

        return Success(value=segments_data)

    def _detect_motion_segments(self, cap: Any, max_frames: int = None) -> List[Tuple[float, float]]:
        """Run the pixel-motion heuristic over ``cap`` and return the detected
        (start, end) play segments.

        Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
        thresholds it into raw segments, then merges dead-time gaps and filters
        out segments shorter than ``min_segment_duration``. ``cap`` is released
        before returning. This is the detection half of ``process_video`` ? it
        performs no DB writes and no fabrication.
        """
        fps = cap.get(cv2.CAP_PROP_FPS)
        total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
        duration = total_frames / fps if fps > 0 else 0

        # Sub-sample settings (5 FPS for analysis)
        analysis_fps = 5.0
        frame_interval = max(1, int(fps / analysis_fps))

        idx_cfg = self.config.get("indexing", {})
        motion_threshold = idx_cfg.get("motion_threshold", 0.08)
        min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
        dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)

        prev_gray = None
        motion_signals = []
        timestamps = []

        frame_idx = 0
        processed_count = 0
        t_start = time.time()
        # Emit a progress line roughly every 5% of the video so long runs are
        # observable instead of silent (a full 1080p match is ~tens of minutes).
        progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
        next_progress = progress_step
        logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
                    f"(~{duration/60:.1f} min), analyzing every {frame_interval}th frame")
        while True:
            # Skip decoding for intermediate frames using cap.grab() for high performance
            for _ in range(frame_interval - 1):
                if not cap.grab():
                    break
                frame_idx += 1

            ret, frame = cap.read()
            if not ret:
                break

            # Process frame
            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            # Apply Gaussian Blur to smooth noise
            gray = cv2.GaussianBlur(gray, (21, 21), 0)

            t = frame_idx / fps
            timestamps.append(t)

            if prev_gray is not None:
                # Frame absolute difference (highly optimized motion tracking)
                diff = cv2.absdiff(prev_gray, gray)
                _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
```

```

                # Dilate to fill gaps
                thresh = cv2.dilate(thresh, None, iterations=2)

                # Motion ratio
                motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] * thresh.shape[1])
                motion_signals.append(motion_ratio)
            else:
                motion_signals.append(0.0)

            prev_gray = gray
            frame_idx += 1
            processed_count += 1

            if progress_step and frame_idx >= next_progress:
                pct = 100.0 * frame_idx / total_frames
                elapsed = time.time() - t_start
                eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
                logger.info(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames, "
                            f"{processed_count} analyzed, {elapsed:.0f}s elapsed, ~{eta:.0f}s left")
                next_progress += progress_step

            if max_frames is not None and processed_count >= max_frames:
                logger.debug(f"Reached max_frames limit of {max_frames}. Halting indexer frame parsing.")
                break

            if frame_idx >= total_frames:
                break

        cap.release()

        # Smooth signal using a moving average
        window_size = 5
        smoothed = []
        for i in range(len(motion_signals)):
            start = max(0, i - window_size // 2)
            end = min(len(motion_signals), i + window_size // 2 + 1)
            smoothed.append(sum(motion_signals[start:end]) / (end - start))

        # Detect active play boundaries
        in_segment = False
        segment_start = None
        raw_segments: List[Tuple[float, float]] = []

        for i, t in enumerate(timestamps):
            signal = smoothed[i]
            if signal > motion_threshold:
                if not in_segment:
                    in_segment = True
                    segment_start = t
            else:
                if in_segment:
                    in_segment = False
                    raw_segments.append((segment_start, t))

        # Handle video ending while in active segment
        if in_segment:
            raw_segments.append((segment_start, timestamps[-1]))

        # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
        merged_segments: List[Tuple[float, float]] = []
```

```
        if raw_segments:
            curr_start, curr_end = raw_segments[0]
            for next_start, next_end in raw_segments[1:]:
                if next_start - curr_end < dead_time_merge_gap:
                    # Merge
                    curr_end = next_end
                else:
                    merged_segments.append((curr_start, curr_end))
                    curr_start, curr_end = next_start, next_end
            merged_segments.append((curr_start, curr_end))

        # Post-Processing: 2. Filter segments shorter than min_segment_duration
        final_segments: List[Tuple[float, float]] = [
            r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
        ]

        return final_segments

    def _populate_db_with_events(self, video_id: str, final_segments: List[Tuple[float, float]],
                                 player_pool: List[str]) -> List[Dict[str, Any]]:
        """Persist ``final_segments`` and their fabricated metadata/events to the DB.

        For each detected segment this assigns a random server/receiver, fabricates
        rich tags, then indexes a serve / 0?3 smash / termination event set ? all
        with the stdlib ``random`` module (the legacy demo's fabricated baseline).
        Returns the per-segment dicts handed back to the caller. The ``random``
        call order here is load-bearing for reproducibility and must not change.
        """
        # Populate SQLite DB with segments and metadata events
        segments_data = []
        for start, end in final_segments:
            # Assign Server and Receiver randomly from pool
            server, receiver = random.sample(player_pool, 2)

            # Formulate dynamic rich tags
            dur = end - start
            metadata = {
                "shot_count": random.randint(4, 25),
                "intensity": "high" if dur > 12 else "medium",
                "avg_speed_kmh": round(random.uniform(40, 85), 1)
            }

            segment_id = self.db.add_play_segment(video_id, start, end, server, receiver, metadata)

            if segment_id:
                # Add sub-segment events
                # A: Serve Event (always at start)
                serve_foul = random.random() < 0.15 # 15% chance of serve foul
                self.db.add_event(
                    segment_id=segment_id,
                    timestamp=start + 0.5,
                    event_type="serve",
                    player=server,
                    details={"status": "foul" if serve_foul else "valid", "type": "low_serve" if random.random() > 0.5 else "high_serve"}
                )

                # B: Smash Events (1 to 3 random smashes per segment)
                smashes_count = random.randint(0, 3)
                for _ in range(smashes_count):
                    smash_time = random.uniform(start + 1.5, end - 1.0)
                    smasher = random.choice([server, receiver])
                    self.db.add_event(
```

```
221                             segment_id=segment_id,
222                             timestamp=round(smash_time, 2),
223                             event_type="smash",
224                             player=smasher,
225                             details={"speed_kmh": round(random.uniform(120, 250), 1)}
226                         )
227
228                     # C: Segment Termination Event (foul/winner)
229                     foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
230                     is_foul = random.random() > 0.4
231                     error_player = random.choice([server, receiver])
232
233                     self.db.add_event(
234                         segment_id=segment_id,
235                         timestamp=end - 0.2,
236                         event_type="foul" if is_foul else "winner",
237                         player=error_player,
238                         details={"type": random.choice(foul_types)} if is_foul else
{"shot_type": "drop" if random.random() > 0.5 else "smash"}
239                     )
240
241                     # Store segment representation to return to caller
242                     segments_data.append({
243                         "id": segment_id,
244                         "start_time": start,
245                         "end_time": end,
246                         "duration": dur,
247                         "server": server,
248                         "receiver": receiver,
249                         "metadata": metadata
250                     })
251
252             return segments_data
253
254     # Register the segmenter
255     segmenter_registry.register(MotionPlaySegmenter)
256
```

41. assistant (2026-06-22T09:51:20.865Z)

Perfect. Now I can confirm: **Motion segmenter always returns `Success(value=segments_data)` where `segments_data` is a list** (possibly empty, but never `None`).

The **real scenario** where `segments_data` could be `None` is:
1. **Motion segmenter** ? returns `Success(value=[])` (empty list if no segments detected) ? Safe
2. **Trajectory-based segmenters** ? return a bare **list** (not wrapped), per line 141-143 comment ? Safe
3. **Hypothetically**, if a future segmenter returns `Success(value=None)` ? that would **crash** at line 350 ? **UNSAFE**

Let me check the base class contract:

42. user (2026-06-22T09:51:21.885Z)

```
1       from abc import ABC, abstractmethod
2       from typing import List, Dict, Any, Type, Optional
3       from backend.infrastructure.database import Database
4       from backend.results import Result  # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6       class BasePlaySegmenter(ABC):
7           def __init__(self, db: Database, config: Dict[str, Any]):
8               self.db = db
9               self.config = config
```

```
    @property
    @abstractmethod
    def name(self) -> str:
        """Unique identifier for this segmenter."""
        pass

    @property
    @abstractmethod
    def description(self) -> str:
        """Human-readable explanation of this segmenter."""
        pass

    @abstractmethod
    def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
        """
        Processes a video to detect play segments, populates SQLite database,
        and returns a Result.

        Use isinstance(result, Failure) (or result.is_success()) to distinguish
        "no segments found" from "something went wrong" (missing key, healthcheck
        failure, etc.). This is the start of the standardized error/result contract.
        """
        pass

class SegmenterRegistry:
    def __init__(self):
        self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}

    def register(self, segmenter_cls: Type[BasePlaySegmenter]):
        """Registers a new segmenter class under its name property."""
        try:
            temp_inst = segmenter_cls(None, {})
            name = temp_inst.name
        except Exception as e:
            raise ValueError(f"Failed to resolve name property from segmenter class {segmenter_cls.__name__}: {e}") from e
        self._segmenters[name] = segmenter_cls

    def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
        return self._segmenters.get(name)

    def list_available(self) -> Dict[str, str]:
        """Returns a dict of registered segmenter names and their descriptions."""
        res = {}
        for name, cls in self._segmenters.items():
            try:
                temp_inst = cls(None, {})
                desc = temp_inst.description
            except Exception:
                desc = "No description available."
            res[name] = desc
        return res

segmenter_registry = SegmenterRegistry()
```

43. assistant (2026-06-22T09:51:34.681Z)

Perfect! The base class declares `Result[List[Dict[str, Any]]]` ? meaning `Success[List[Dict[str, Any]]]` (where the value is always a list, never `None`).

ANALYSIS CONCLUSION:

The finding is **technically accurate but MISLEADING**:

1. **The stated concern** ("line 350 calls _segments_to_intervals(segs) unconditionally even when segs is None") is **NOT a real path** because:
   - Motion segmenter returns `Success(value=segments_data)` where `segments_data` is always a list
   - Trajectory segmenters return a bare **list** (not wrapped)
   - The base class contract declares `Result[List[...]]` ? the value type is always `List`, never `None`

2. **However, there IS a subtle gap**: The base class is declared but NOT enforced. The trajectory segmenters return bare lists, not wrapped `Success` objects. This is documented as intentional (line 141-143), but it's a **latent type contract violation** that could bite if:
   - A future segmenter mistakenly returns `Success(None)` or `Success(value=None)`
   - The code would crash with `TypeError: 'NoneType' object is not iterable` at line 350
   - This would NOT be caught by the early error-check (line 347) because it's a Success, not a Failure

3. **The rerun guard (line 362-366)** is correctly placed and defensive for **actual DB-drop transients**, not for this hypothetical crash.

VERDICT: The finding's assessment "No actual bug, but the logic path is subtle" is **TRUE but incomplete**:
- ? No bug in the **current actual code** (segmenters follow the contract)
- ? The None-handling rerun guard is correct
- ? But there IS a **latent safety gap**: if a segmenter violates the contract by returning `Success(None)`, line 350 would crash unhandled

Let me verify if there's any defensive code that should catch this:

44. user (2026-06-22T09:51:36.320Z)

```
309     def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
310                       rerun_on_mismatch: bool = True,
311                       baseline_count: Optional[int] = None) -> Dict[str, Any]:
312         """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.
313
314         The re-run-once guard (the one known non-determinism ? a transient
``add_play_segment``?None drop,
315         database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
316         reproduce, a real change will."""
317         import random
318         import time
319
320         from backend.eval import rally_seg_eval
321         from backend.infrastructure.database import Database
322         from backend.pipeline.segmenters import segmenter_registry
323         from backend.results import Failure
324         from backend.storage import fetch
325         from backend.utils.hashing import compute_video_id
326
327         name = video["name"]
328         try:
```

45. assistant (2026-06-22T09:51:38.434Z)

The entire `run_one_video` function is inside a `try-except` context. Let me check:

46. user (2026-06-22T09:51:39.616Z)

<tool_use_error>InputValidationError: Read was called with input that could not be parsed as JSON.

You sent (first 157 of 157 bytes): {"file_path": "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/scripts/nightly_reelcount.py", "offset": ::327, "limit": 85}
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

47. user (2026-06-22T09:51:42.478Z)

```
        name = video["name"]
        try:
            local = fetch(video["uri"])
        except Exception as e:  # noqa: BLE001 ? surface, never hide
            return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed: {e}",
                    "quality": None, "wall_seconds": 0.0}

        gts = _load_gts(video.get("labels_uri"))
        seg_cls = segmenter_registry.get(segmenter)
        if not seg_cls:
            return {"name": name, "reel_count": None, "ok": False,
                    "error": f"segmenter '{segmenter}' not registered", "quality": None, "wall_seconds": 0.0}

        def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float, float]]], float, Optional[str]]:
            random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields are reproducible too
            db = Database(db_path)
            video_id = compute_video_id(local)
            t0 = time.monotonic()
            result = seg_cls(db, config).process_video(video_id, local)
            wall = time.monotonic() - t0
            if isinstance(result, Failure):
                return None, None, wall, getattr(result, "message", "segmenter failed")
            segs = result.value if hasattr(result, "value") else result
            intervals = _segments_to_intervals(segs)
            return len(segs), intervals, wall, None

        import tempfile
        total_wall = 0.0
        with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
            count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
            total_wall += wall
            if err is not None:
                return {"name": name, "reel_count": None, "ok": False, "error": err,
                        "quality": None, "wall_seconds": round(total_wall, 2)}
            # re-run-once guard for the transient DB-drop flake
            if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
                count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
                total_wall += wall2
                if err2 is None and count2 is not None:
                    count, intervals = count2, intervals2  # the reproduced (or corrected) value

            quality = None
            if gts and intervals is not None:
                row = rally_seg_eval.score_intervals(intervals, gts)
                quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25", "tol_precision", "tol_recall")
                           if k in row}

        return {"name": name, "reel_count": count, "ok": True, "error": None,
                "quality": quality, "wall_seconds": round(total_wall, 2)}
```

```
def load_manifest(path: str) -> Dict[str, Any]:
    with open(path, encoding="utf-8") as fh:
        return json.load(fh)


def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
               rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
    """Run every manifest video through the configured pipeline. Returns (run_results,
total_wall_s)."""
    from backend.config import load_config
```

48. assistant (2026-06-22T09:51:46.536Z)

KEY FINDING: The `_process()` function at line 340 is **NOT wrapped in a try-except**. If line 350 crashes with `TypeError` (because `segs is None`), the exception will propagate **uncaught** from `_process()`.

But line 356's call to `_process()` is inside the outer function, which means the exception will eventually bubble up to `run_corpus()` at line 486, which IS in a try-except:

49. user (2026-06-22T09:51:48.221Z)

```
485         try:
486             run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
487         except Exception as e:  # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
488             log.error("run failed: %s", e)
489             return 2
490         if not run_results:
491             log.error("no videos in manifest %s", args.manifest)
492             return 2
493
494         billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
```

50. assistant (2026-06-22T09:51:57.595Z)

So **any crash in `_process()` would be caught by the outer exception handler at line 487**, which would:
1. Log the error
2. Return exit code 2 (operational failure, not a false pass)

This is **defensive**, but the problem is:
- Line 350 would crash with `TypeError: 'NoneType' object is not iterable` if `segs is None`
- This would be caught as a generic "operational failure" instead of being reported as a segmenter contract violation
- The error message would be opaque to the user

FINAL VERDICT:

The finding's claim is **TECHNICALLY CORRECT but the risk assessment is OPTIMISTIC**. Here's the truth:

1. **No bug TODAY** ? all current segmenters follow the contract (they return `List`, never `None`)
2. **LATENT SAFETY GAP** ? if a segmenter violates the contract by returning `Success(None)` or `Success(value=None)`:
   - Line 350 crashes with `TypeError`
   - The crash is caught as a generic operational failure (exit code 2)
   - The error reporting is opaque (not distinguishable from other IO failures)
3. **The rerun guard is NOT relevant** ? it only handles DB transients, not type contract violations

RECOMMENDATION: Add defensive type-checking at line 349 to catch and handle the edge case explicitly:

```python
segs = result.value if hasattr(result, "value") else result
if not isinstance(segs, list):
    return None, None, wall, f"segmenter returned invalid type: {type(segs).__name__} (expected list)"
```

This is a **LOW SEVERITY** but **REAL defect** ? it's a defensive gap that could mask a regression if a segmenter implementation regresses.

Now let me prepare the final report:

51. user (2026-06-22T09:52:07.581Z)

Structured output provided successfully

52. assistant (2026-06-22T09:52:17.358Z)

Summary

I've completed an **ADVERSARIAL VERIFICATION** of the provided finding. Here's the verdict:

The finding is REAL, but not in the way described.

What I Found

The stated claim: "If isinstance(result, Failure) at line 347, the function returns early at line 348. But if result is a Success with .value=None (a malformed segmenter), or result is a bare list=[], line 350 _segments_to_intervals([]) is called safely."

The assessment: "No actual bug, but the logic path is subtle."

VERDICT ? INCORRECT OPTIMISM:

While the logic path described is technically accurate for the **current code** (all existing segmenters follow the contract), there IS a **real latent defect**:

1. **The base class contract** (backend/pipeline/segmenters/base.py:24) declares `Result[List[Dict[str, Any]]]` ? the value must always be a List.

2. **If a segmenter returns `Success(None)` or `Success(value=None)`** (violating the contract):
   - Line 350 crashes with `TypeError: 'NoneType' object is not iterable`
   - The crash is **unhandled** at the `_process()` function level
   - It bubbles up as a generic "operational failure" (exit 2)
   - The error message is **opaque** instead of reporting a contract violation

3. **Today there's no production bug** because motion.py returns `Success(list)` and trajectory segmenters return bare lists ? but this is **not enforced**.

4. **The rerun guard (line 362-366)** is irrelevant ? it only handles DB transients (`add_play_segment` returning None), not type contract violations.

Recommended Fix

Add type validation at line 349:
```python
if not isinstance(segs, list):
    return None, None, wall, f"segmenter returned invalid type: {type(segs).__name__} (expected list)"
```

```

This catches segmenter regressions explicitly instead of masking them as opaque operational failures.
```